

Logistic Regression: Gradient Descent and Code from Scratch

Contents

1. Overview	2
2. Predictions in matrix form	2
2.1 One prediction	2
2.2 All predictions together	2
3. The loss in matrix form	3
4. The gradient	4
4.1 The first part	4
4.2 The second part	4
4.3 Putting them together	4
5. The update rule	5
6. The code	6
7. Checking against scikit-learn	7
8. Perfectly separable data	8
9. Summary	10
10. Sources	10
11. Key terms	10

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Classification problems ([Note 3](#)); Model-based learning ([Note 6](#)); Gradient descent ([Note 57](#)); Polynomial features ([Note 61](#)); Perceptron trick ([Note 71](#)); Log loss (binary cross entropy) ([Note 73](#)).
- **Leads to:** Softmax regression ([Note 79](#)).
- **Compare with:** Support vector machines ([Note 92](#)).

1. Overview

Key point: The gradient of the log loss is $-(1/m) X^T(y - \hat{y})$. Gradient descent repeats $w \leftarrow w + \eta (1/m) X^T(y - \hat{y})$, and reaches the same answer as scikit-learn.

The log loss Note gave logistic regression its loss function and noted that it has no closed-form minimum. This Note finishes the journey:

1. write the predictions and the loss in matrix form;
2. differentiate the loss with respect to the **weights** (G-2111), the numbers the model learns;
3. code gradient descent and check it against scikit-learn.

2. Predictions in matrix form

Key point: All m predictions at once: $\hat{y} = \sigma(Xw)$, where X has a first column of 1s.

2.1 One prediction

Key point: For one observation, $\hat{y} = \sigma(w_0 + w_1x_1 + \dots + w_nx_n)$.

Take a dataset with m **observations** (G-1374) (records, one row of the data table each) and n **features** (G-772) (input variables, one column each). The **target** (G-1949) y is the output we predict, here the class 0 or 1. The model has $n + 1$ weights: w_0 (the **intercept** (G-960), the part of the prediction that does not depend on any feature) and w_1 to w_n . The prediction for observation i is

$$\hat{y}_i = \sigma(w_0 + w_1x_{i1} + w_2x_{i2} + \dots + w_nx_{in})$$

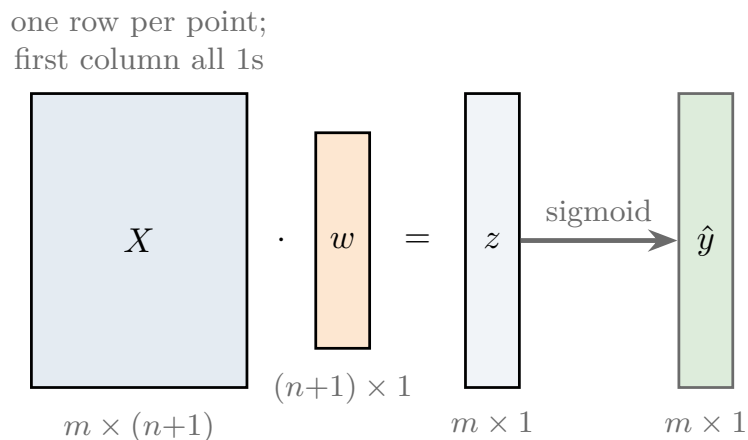
2.2 All predictions together

Key point: Stack the rows: the inner sums are the matrix product Xw .

Writing out all m predictions, each inner sum is one row of X (with a 1 added in front) times the weight vector w . So all of them together are a single matrix product:

$$\hat{y} = \sigma(Xw)$$

where σ is applied to each entry. Stacking the predictions as Xw is the same trick as in multiple linear regression.



$$\frac{\partial L}{\partial w} = -\frac{1}{m} X^T (y - \hat{y})$$

X^T is $(n+1) \times m$, $(y - \hat{y})$ is $m \times 1$:
the gradient is $(n+1) \times 1$, one slope
per weight, the same shape as w

Figure 1: The shapes of X , w , $\hat{y}$ and the gradient

In Figure 1, read the top row left to right: the m rows of X times the one column w give one z per observation, and the sigmoid turns each z into a prediction.

3. The loss in matrix form

Key point: $L = -(1/m)[y^T \log \hat{y} + (1 - y)^T \log(1 - \hat{y})]$.

The log loss from the previous Notes is

$$L = -\frac{1}{m} \sum_{i=1}^m [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

The sum $\sum_i y_i \log \hat{y}_i$ multiplies matching entries of two vectors and adds them up, which is the **dot product** (G-634) $y^T \log \hat{y}$. So

$$L = -\frac{1}{m} \left[y^\top \log \hat{y} + (1 - y)^\top \log(1 - \hat{y}) \right], \quad \hat{y} = \sigma(Xw)$$

4. The gradient

Key point: Differentiate each part with the chain rule; the sigmoid derivative cancels the logs, leaving $-(1/m) X^\top (y - \hat{y})$.

4.1 The first part

Key point: The derivative of $y \log \hat{y}$ with respect to w is $y(1 - \hat{y}) x$.

For a single observation, differentiate $y \log \hat{y}$ step by step with the **chain rule** (G-371) (multiply the derivatives of the nested steps):

- the derivative of $\log \hat{y}$ with respect to $\hat{y}$ is $1/\hat{y}$;
- the derivative of $\hat{y} = \sigma(z)$ with respect to z is $\hat{y}(1 - \hat{y})$ (the sigmoid derivative Note);
- the derivative of $z = w \cdot x$ with respect to w is x .

Multiplying: $y \cdot \frac{1}{\hat{y}} \cdot \hat{y}(1 - \hat{y}) \cdot x = y(1 - \hat{y}) x$. The $\hat{y}$ cancels.

4.2 The second part

Key point: The derivative of $(1 - y) \log(1 - \hat{y})$ is $-(1 - y) \hat{y} x$.

In the same way, for $(1 - y) \log(1 - \hat{y})$:

- the derivative of $\log(1 - \hat{y})$ with respect to $\hat{y}$ is $-1/(1 - \hat{y})$;
- the rest of the chain is the same: $\hat{y}(1 - \hat{y}) x$.

Multiplying: $(1 - y) \cdot \frac{-1}{1 - \hat{y}} \cdot \hat{y}(1 - \hat{y}) \cdot x = -(1 - y) \hat{y} x$.

4.3 Putting them together

Key point: $y(1 - \hat{y}) - (1 - y)\hat{y}$ simplifies to $y - \hat{y}$.

Adding the two parts:

$$y(1 - \hat{y}) - (1 - y)\hat{y} = y - y\hat{y} - \hat{y} + y\hat{y} = y - \hat{y}$$

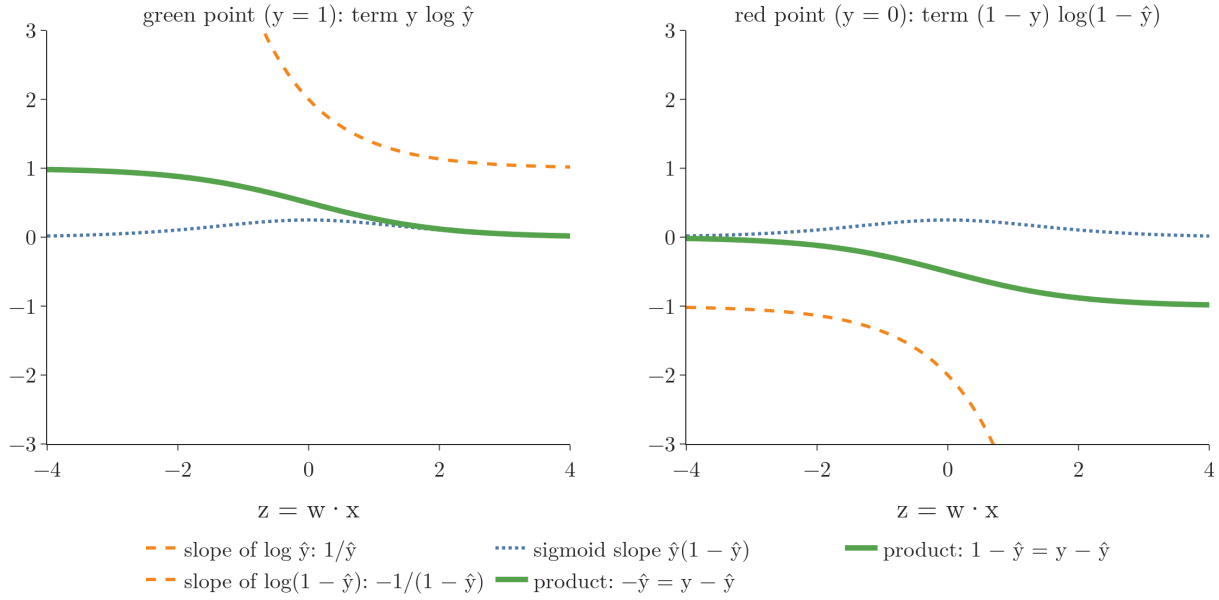


Figure 2: The cancellation along z . The log’s slope (orange, dashed) blows up where $\hat{y}$ nears 0 or 1, and the sigmoid’s slope (blue, dotted) shrinks there. Their product (green) is the plain $y - \hat{y}$.

In Figure 2, watch the green product stay between -1 and 1 even where the orange slope shoots off the chart: the two chain-rule factors cancel exactly.

So for one observation, the derivative of the bracket is $(y - \hat{y})x$. Summing over all observations, with the $-\frac{1}{m}$ in front, and writing the sum as a matrix product:

$$\frac{\partial L}{\partial w} = -\frac{1}{m} X^\top (y - \hat{y})$$

$X^\top$ has shape $(n + 1) \times m$ and $(y - \hat{y})$ has shape $m \times 1$, so the **gradient** (G-865) has shape $(n + 1) \times 1$: one slope per weight, the same shape as w (Figure 1).

Extra: This gradient has exactly the same form as the one for linear regression with mean squared error: $X^\top(\text{actual} - \text{predicted})$, scaled. The only difference is that the predictions now pass through the sigmoid (Bishop §4.3.2).

5. The update rule

Key point: $w_{\text{new}} = w_{\text{old}} + \eta (1/m) X^\top (y - \hat{y})$.

Gradient descent steps against the gradient. The step size η is the **learning rate** (G-1068):

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w} = w_{\text{old}} + \frac{\eta}{m} X^\top (y - \hat{y})$$

Compare it with the sigmoid perceptron of the sigmoid Note: $w \leftarrow w + \eta(y - \hat{y})x$ for one random point. The new rule is the same idea, averaged over **all** points in each step. The perceptron was one-point **stochastic gradient descent** (G-1892) on this very loss, run for a fixed number of steps. **Batch gradient descent** (G-264) uses all points in every step and, run to the end, reaches the true minimum.

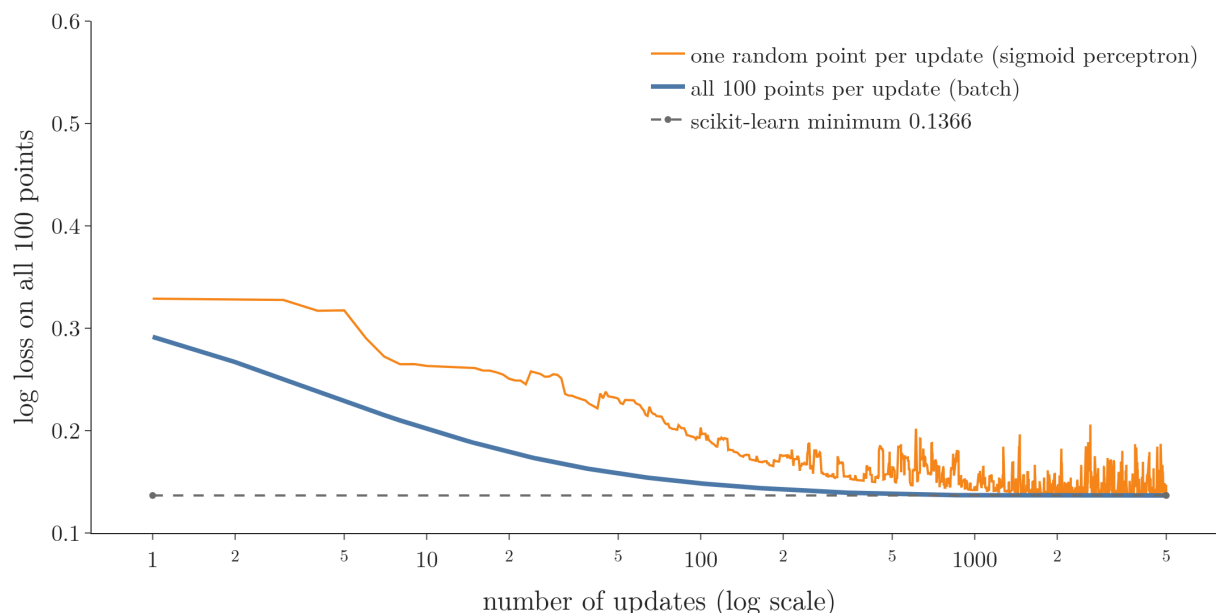


Figure 3: Log loss on the 100 points of Section 7 against the number of updates, both starting from $w = (1, 1, 1)$. Orange: the sigmoid perceptron’s rule, one random point per update (learning rate 0.1). Blue: the batch rule, all points per update (learning rate 0.5).

In Figure 3, watch the orange curve jitter above the dashed minimum (0.1383 after 5,000 updates) while the blue curve settles onto it (0.1366): each one-point step pulls towards a single point, the batch step towards all of them.

6. The code

Key point: Add a column of 1s, then repeat: predict with the sigmoid, and update with $X^T(y - \hat{y}) / m$.

Python: Logistic regression with batch gradient descent.

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def gd(X, y, lr=0.5, epochs=5000):
    X = np.insert(X, 0, 1, axis=1)           # column of 1s
    w = np.ones(X.shape[1])
    for _ in range(epochs):
```

```

y_hat = sigmoid(X @ w)                                # all predictions
w = w + lr * (X.T @ (y - y_hat)) / X.shape[0]
return w[1:], w[0]                                     # coef_, intercept_

```

7. Checking against scikit-learn

Key point: After 5,000 epochs, the weights match scikit-learn's to three decimals.

One **epoch** (G-696) is one pass over all the training points; batch gradient descent makes one update per epoch.

The test data has 100 points with two features whose classes overlap a little (`make_classification, class_sep=1.5, random state 4`). scikit-learn's `LogisticRegression(penalty=None)` fits the same model without regularisation, so the two should agree.

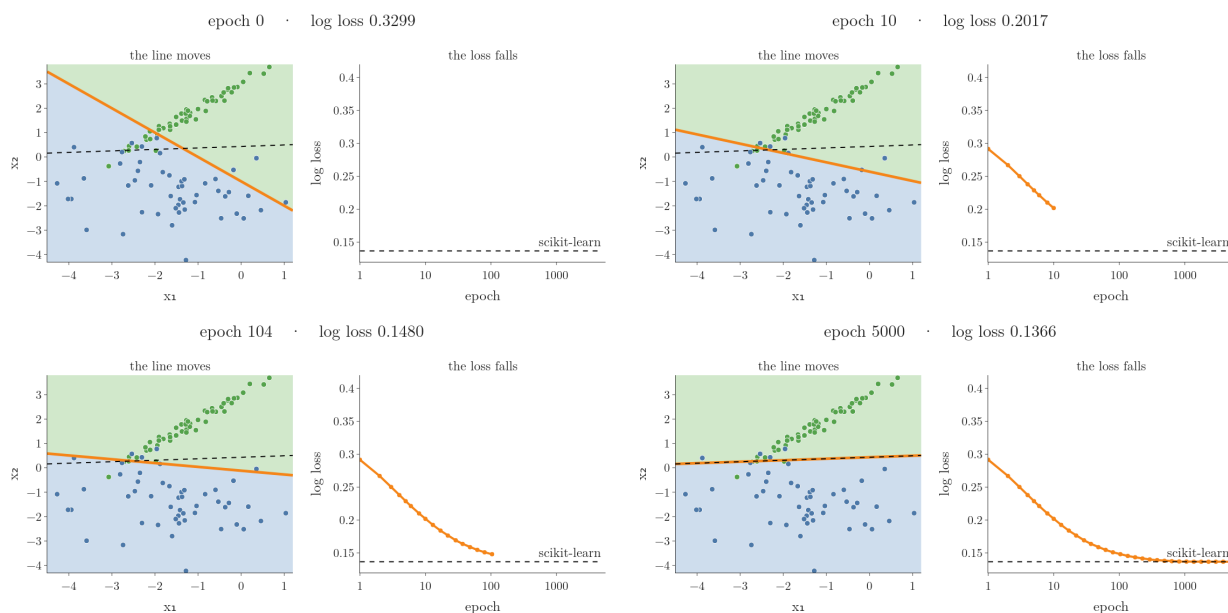


Figure 4: Batch gradient descent on the 100 points. Left: the decision boundary ($p = 0.5$) turns into place; the dashed line is scikit-learn's. Right: the log loss per epoch (log scale) falls to scikit-learn's minimum.

In Figure 4, watch the orange line, the **decision boundary** (G-555) where $p = 0.5$, swing from its start at $w = (1, 1, 1)$ onto scikit-learn's dashed decision boundary while the loss curve flattens onto the dashed minimum.

	Intercept	w_1	w_2	Log loss
Start	1	1	1	
After 100 epochs	0.360	0.454	2.850	0.1484
After 1,000 epochs	-1.227	-0.094	3.542	0.1368

	Intercept	w_1	w_2	Log loss
After 5,000 epochs	-1.568	-0.220	3.623	0.1366
scikit-learn	-1.569	-0.220	3.623	0.1366

The loss falls quickly at first and then levels off at scikit-learn's minimum (Figure 4, right). The decision boundary turns into place and ends on top of scikit-learn's (left). The model classifies 92% of the points correctly; the rest lie in the overlap, where no straight decision boundary can be perfect.

Python: The scikit-learn equivalent.

```
from sklearn.linear_model import LogisticRegression

lor = LogisticRegression(penalty=None)    # no regularisation
lor.fit(X, y)
lor.intercept_, lor.coef_                # -1.569, [-0.220, 3.623]
```

Extra: By default, `LogisticRegression` adds an L2 (Ridge) penalty with `C=1`, so its weights are slightly smaller than these. `penalty=None` switches it off to match our code; the hyperparameters Note covers the options.

8. Perfectly separable data

Key point: If a straight decision boundary separates the classes perfectly, the loss can always be made smaller by making the weights larger, so the weights never settle.

On the data of the perceptron Notes, where the classes are perfectly separated (`class_sep=20`), something different happens. This case is called **perfect separation** (G-1487):

Epochs	Weights (w_0, w_1, w_2)	Log loss
500	(3.63, 3.35, 0.11)	0.0052
5,000	(5.83, 4.84, 0.21)	0.00063
50,000	(8.20, 6.52, 0.37)	0.00007

The weights keep growing, and the loss keeps shrinking towards 0 without ever reaching it.

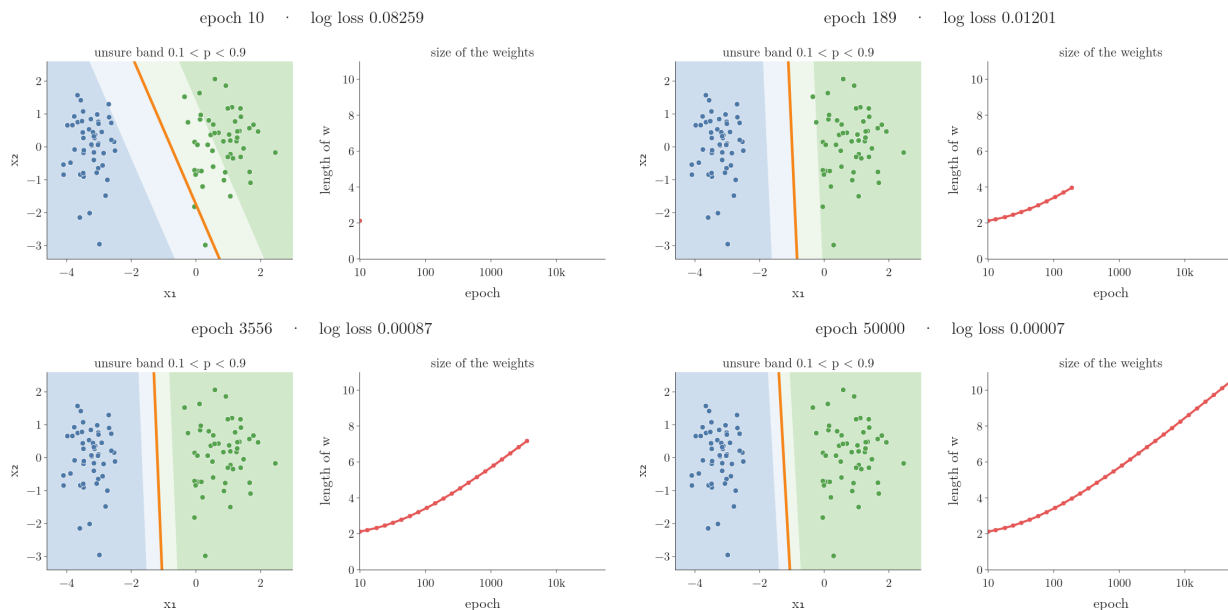


Figure 5: Batch gradient descent on perfectly separated points. Left: the pale band is where the model is unsure ($0.1 < p < 0.9$); the orange line is the decision boundary, $p = 0.5$. Right: the length of the weight vector per epoch (log scale).

In Figure 5, the decision boundary turns only slowly, yet the weights keep growing and the unsure band keeps narrowing: the model grows ever more confident instead of settling.

The reason: once every point is on its correct side, multiplying all the weights by 2 keeps the same decision boundary but makes every z twice as large, so every $\hat{y}$ moves closer to its correct 0 or 1. The loss always decreases, so there is no finite minimum (Bishop §4.3.2). The decision boundary itself still turns slowly: its slope $-w_1/w_2$ goes from -30 to -23 to -18 in the table. Gradient descent turns it towards the decision boundary with the widest gap, but only very slowly (Soudry et al. 2018).

In practice this is handled by stopping after a fixed number of epochs, or by adding **regularisation** (G-1659), a penalty on large weights in the loss, the standard fix (Bishop §4.3.2). scikit-learn uses a penalty by default.

9. Summary

- Predictions: $\hat{y} = \sigma(Xw)$.
- Loss: $L = -\frac{1}{m}[y^\top \log \hat{y} + (1 - y)^\top \log(1 - \hat{y})]$.
- Gradient: $\frac{\partial L}{\partial w} = -\frac{1}{m}X^\top(y - \hat{y})$, thanks to $\sigma' = \sigma(1 - \sigma)$.
- Update: $w \leftarrow w + \frac{\eta}{m}X^\top(y - \hat{y})$; the code matches `LogisticRegression(penalty=None)`.
- On perfectly separable data the weights grow without limit; regularisation or early stopping keeps them finite.

10. Sources

Built from

- CampusX, “Logistic Regression Part 5 | Gradient Descent & Code From Scratch”, YouTube, <https://www.youtube.com/watch?v=ABrrSwMYWSg>

Other references

- **Bishop:** Bishop, C. M. *Pattern Recognition and Machine Learning*. Springer, 2006. Section 4.3.2, pp. 205–207.
- **Soudry et al. 2018:** Soudry, D., Hoffer, E., Nacson, M. S., Gunasekar, S. and Srebro, N. “The Implicit Bias of Gradient Descent on Separable Data.” *Journal of Machine Learning Research* 19(70), 1–57, 2018.

11. Key terms

Term	Meaning
Decision boundary	The line where the model’s probability is exactly 0.5; one class is predicted on each side
Learning rate (η)	The step size η that scales each gradient descent update
Epoch	One pass over all the training points
Gradient	The vector of slopes of the loss, one for each weight
Batch gradient descent	Gradient descent that uses all observations for every update
penalty=None	LogisticRegression setting that switches regularisation off
Perfect separation	When a straight decision boundary splits the training classes with no mistakes; unregularised weights then grow without limit